Proyecto final de servicios MVC y broker en la nube

Olyver Mark Jason Osorio Tojín
Facultad de Ingeniería
Universidad Mesoamericana (UMES) – Sede Quetzaltenango
Cátedra: Arquitectura de Sistemas I (Proyecto Final)

I. INTRODUCCIÓN

El presente proyecto final de la cátedra de Arquitectura de Sistemas I consiste en el diseño, desarrollo, despliegue y validación de una plataforma reducida de comercio electrónico basada en un ecosistema de microservicios independientes. Siguiendo los requerimientos del enunciado, el sistema no cuenta con ninguna interfaz de usuario (*frontend*), por lo que toda la interacción y verificación se realiza directamente a nivel de API mediante Postman.

La arquitectura exigida contempla tres componentes desacoplados:

1. Un **Broker/Mediador** que funciona como único punto de entrada al sistema con autenticación por token, sin contener lógica de negocio.
2. Un **Servicio de Productos** implementado bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC), con base de datos propia y validaciones de catálogo.
3. Un **Servicio de Pedidos** estructurado en una Arquitectura en Capas, con su propia base de datos, el cual consulta la existencia del producto y solicita el descuento comunicándose obligatoriamente a través del broker.

Para dar solución a este requerimiento, el proceso se abordó en dos etapas principales:

- **Etapa en local:** Se diseñaron los tres microservicios en TypeScript con NestJS y TypeORM sobre bases de datos MySQL independientes. Se orquestaron localmente con Docker Compose, garantizando el aislamiento de red y validando el funcionamiento del broker, las reglas de negocio y las pruebas unitarias con *mocks*.
- **Etapa en la nube:** En lugar de utilizar servicios gestionados comerciales más costosos y complejos como AWS o Azure, se optó por aprovisionar un Servidor Privado Virtual (VPS) propio con Linux y K3s. Se configuró un subdominio público (`api-tienda.soymythpick.com`) administrado mediante Cloudflare DNS, con enrutamiento Ingress en Traefik y certificados SSL/TLS emitidos automáticamente por Let's Encrypt.

A través de este flujo completo, se logró una solución robusta, segura y completamente verificable mediante colecciones automatizadas en Postman y Newman.

II. ARQUITECTURA DE LOS SERVICIOS

A. Broker / Mediador (Puerto 3000)

El Broker actúa como la compuerta perimetral del sistema. No contiene lógica de negocio; su propósito principal es recibir las solicitudes externas, verificar la validez del token JWT en la cabecera `Authorization: Bearer` y reenviar la petición al microservicio que corresponda (`productos` o `pedidos`).

Para la comunicación interna hacia los servicios protegidos, el Broker inyecta un token de firma de servicio (`SERVICIO_TOKEN`), impidiendo que clientes externos interactúen directamente con los servicios de catálogo o transacciones. Si una petición carece de credenciales o el token no es válido, el Broker rechaza la solicitud de inmediato con código HTTP 401 Unauthorized.

B. Servicio de Productos (Puerto 3001)

Encargado de la gestión de productos e inventario. Está estructurado bajo el patrón **Modelo-Vista-Controlador (MVC)**:

- **Modelo:** Entidad `Producto` mapeada con TypeORM, que registra código, nombre, precio unitario y existencias en stock.
- **Vista:** DTOs y clases de presentación REST (`ProductoVista`, `DescuentoVista`) que moldean las respuestas JSON enviadas al cliente.
- **Controlador:** Expone endpoints para el CRUD de productos, validaciones de negocio (precio > 0, stock ≥ 0) y la ruta protegida utilizada por el servicio de pedidos para aplicar descuentos de inventario.

C. Servicio de Pedidos (Puerto 3002)

Maneja el registro y confirmación de las órdenes de compra bajo una **Arquitectura en Capas**:

- **Capa de Presentación:** Controladores que reciben los parámetros del pedido y validan los datos de entrada.
- **Capa de Negocio / Dominio:** Contiene la lógica del pedido. Antes de confirmar una compra, consulta al microservicio de productos a través del Broker para verificar la existencia del artículo y la disponibilidad del stock. Si la validación es exitosa, solicita el descuento y confirma la orden; de lo contrario, registra el rechazo. Además, controla fallos de comunicación y tiempos de espera agotados (*timeout*).
- **Capa de Datos:** Repositorio TypeORM que persiste las órdenes con su identificador UUID, estado (`PENDIENTE`, `CONFIRMADO`, `RECHAZADO`) y fecha de registro.

D. Persistencia Independiente

Cada uno de los tres servicios cuenta con su propia base de datos dentro de MySQL 8.4: `broker`, `productos` y `pedidos`. Cada esquema dispone de un usuario SQL con credenciales independientes y permisos limitados a su base de datos, garantizando el desacoplamiento de datos exigido.

III. PUESTA EN MARCHA Y PRUEBAS EN ENTORNO LOCAL

A. Orquestación con Docker Compose

Para la fase de desarrollo y verificación local se elaboró el archivo `compose.yaml`, el cual inicia de manera coordinada la base de datos MySQL, el Broker, Productos y Pedidos en una red privada denominada `tienda`.

A nivel de red, únicamente el Broker expone su puerto hacia el equipo local (`127.0.0.1:3000:3000`). Los servicios de Productos (puerto 3001) y Pedidos (puerto 3002) operan de forma confinada dentro de la red interna de Docker, garantizando que ninguna llamada externa pueda eludir la mediación del Broker.

La Fig. 1 muestra la recreación y el levantamiento de los contenedores en terminal junto a la verificación de estado con `docker compose ps`, confirmando que los cuatro contenedores se encuentran activos y en ejecución normal.

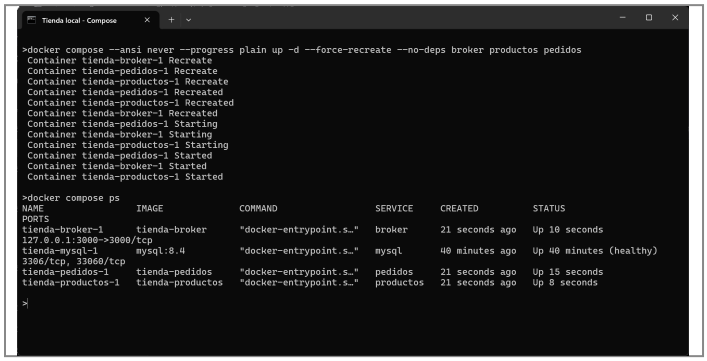


Fig. 1. Consola de comandos evidenciando la recreación de los servicios y el estado de los contenedores mediante `docker compose ps`.

Por su parte, la Fig. 2 presenta la interfaz gráfica de Docker Desktop en la sección *Containers*, donde se aprecia el grupo `tienda` desplegado con MySQL, Broker, Pedidos y Productos operando correctamente con sus respectivos puertos.

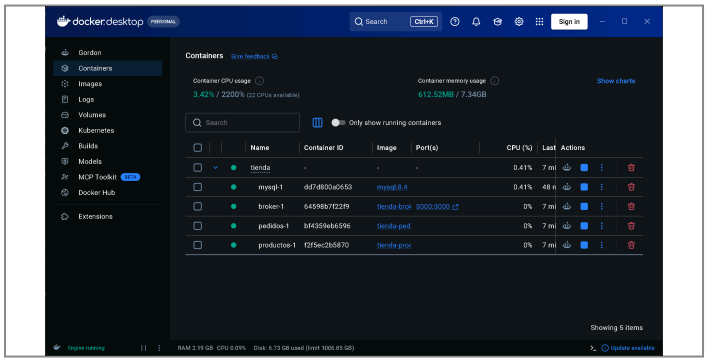


Fig. 2. Panel de Docker Desktop con el grupo de contenedores `tienda` expandido y en ejecución en entorno local.

B. Pruebas de Seguridad y Pruebas Unitarias

En el entorno local se realizaron pruebas con Postman contra la dirección `http://localhost:3000`. Como se observa en la Fig. 3, al intentar consultar el catálogo en `/productos` sin incluir credenciales de autenticación, el Broker intercepta la petición y responde con código HTTP **401 Unauthorized** y el mensaje descriptivo `"Token ausente, inválido o expirado"`.

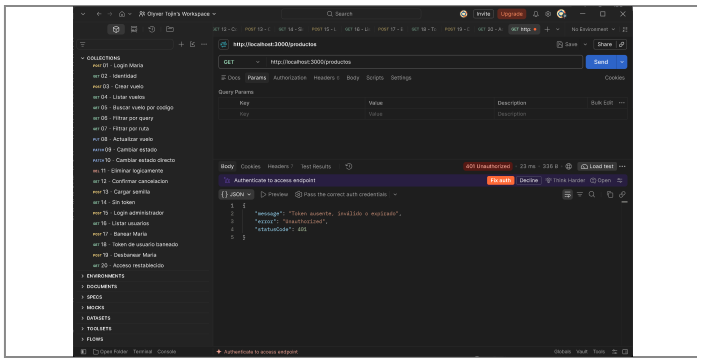


Fig. 3. Prueba en Postman contra `http://localhost:3000/productos` sin token, evidenciando el rechazo 401 Unauthorized.

Asimismo, se implementaron pruebas unitarias automatizadas con *mocks* utilizando el módulo nativo `node:test` de Node.js, cubriendo la confirmación de pedidos con descuento exitoso, el rechazo por inventario insuficiente (`ConflictException`) y el manejo de tiempos de espera (`GatewayTimeoutException`).

IV. DESPLIEGUE EN SERVIDOR VIRTUAL (VPS) Y CONFIGURACIÓN DNS

Para la publicación del proyecto en la nube, en lugar de recurrir a plataformas como AWS o Google Cloud, se decidió montar la infraestructura en un Servidor Privado Virtual (VPS) propio con Ubuntu 24.04 LTS, aprovechando la distribución ligera **K3s** de Kubernetes.

A. Configuración del Dominio en Cloudflare

Se utilizó el dominio propio `soymythpick.com` gestionado a través de Cloudflare. Para dirigir las peticiones hacia el VPS, se creó un registro DNS de tipo **A** para el subdominio `api-tienda.soymythpick.com` apuntando a la dirección IP pública del servidor (`207.244.247.32`).

El registro se estableció en modo *DNS only* (resolución directa sin proxy intermedio), como se documenta en el panel de Cloudflare mostrado en la Fig. 4. Esto permitió que la terminación TLS y los desafíos de validación de certificados se gestionaran directamente en el servidor.

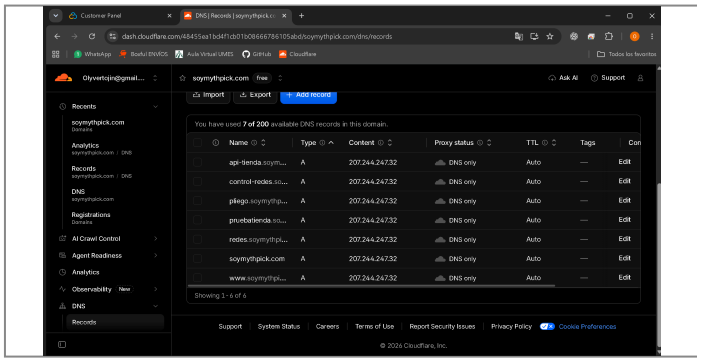


Fig. 4. Panel de Cloudflare confirmando el registro A de `api-tienda.soymythpick.com` apuntando a la IP pública del VPS.

B. Despliegue con K3s, Traefik y Certificados SSL

Dentro del VPS, se organizaron los servicios dentro de un espacio de nombres (*namespace*) denominado `tienda`. La arquitectura en el clúster se configuró de la siguiente forma:

- Los microservicios (`broker`, `productos`, `pedidos` y `mysql`) corren como *Deployments* independientes con servicios de

tipo `ClusterIP`, sin exponer puertos directos al exterior.

- Se empleó el controlador **Traefik Ingress** predeterminado de K3s para canalizar el tráfico web. Se configuró una regla de redirección automática permanente de HTTP a HTTPS.
- Se integró **Cert-Manager** junto con Let's Encrypt para generar y renovar automáticamente certificados SSL/TLS válidos para el subdominio.

La Fig. 5 muestra la terminal de administración del VPS (*Tienda VPS - K3s*), donde se certifica que los cuatro pods se encuentran en estado *Running*, los servicios ClusterIP asignados, los recursos Ingress operando sobre la IP `207.244.247.32` y el certificado TLS emitido con estado `READY: True`.

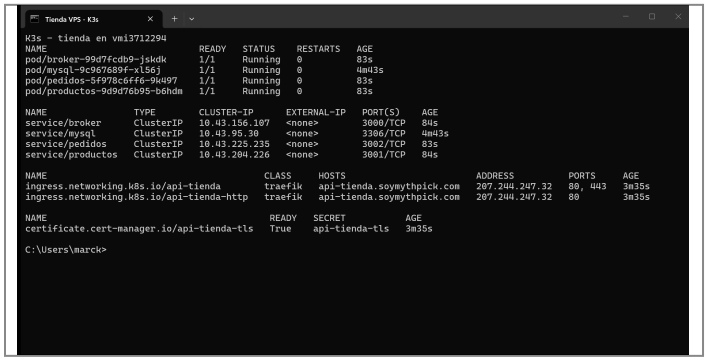


Fig. 5. Estado del clúster K3s en el VPS mostrando pods, servicios, ingress y certificado TLS activos en el namespace `tienda`.

V. VALIDACIÓN INTEGRAL DE LA API EN LA NUBE

Una vez levantado y securizado el entorno en el VPS, se ejecutó la colección completa de pruebas `tienda-vps.postman_collection.json` de forma desatendida mediante el corredor de consola **Newman**.

Las pruebas se ejecutaron directamente contra la URL pública segura `https://api-tienda.soymythpick.com`. Como se evidencia en la salida de terminal documentada en la Fig. 6, se completaron exitosamente:

- **16 peticiones HTTP/HTTPS ejecutadas.**
- **19 aserciones comprobadas.**
- **0 fallos detectados.**

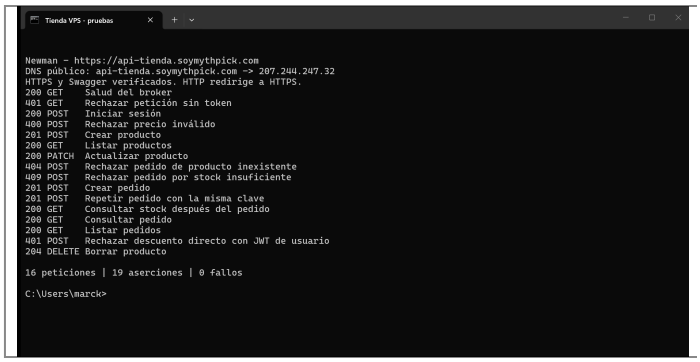


Fig. 6. Ejecución de la colección de Postman mediante Newman en la nube, con 16 peticiones, 19 aserciones y 0 fallos.

El ciclo evaluado cubrió la verificación del endpoint de diagnóstico `GET /salud`, el rechazo de peticiones anónimas, el inicio de sesión con emisión de JWT, validaciones de negocio de precios y stock, creación y consulta de productos con MVC, confirmación de pedidos con descuento de existencias a través del broker, pruebas de idempotencia con claves repetidas y el rechazo de intentos de descuento no autorizados.

VI. CONCLUSIONES

La solución implementada da respuesta completa a todos los requerimientos estipulados para el proyecto final de Arquitectura de Sistemas I. Se logró desacoplar satisfactoriamente las responsabilidades del sistema empleando el patrón Broker como mediador perimetral de autenticación, el patrón MVC para el catálogo de productos y una arquitectura en capas para la gestión de pedidos, manteniendo bases de datos MySQL independientes por servicio.

Asimismo, la progresión desde el entorno local orquestado con Docker Compose hacia un despliegue en producción en un VPS propio con K3s, subdominio en Cloudflare y certificados SSL/TLS automáticos, demostró que es viable construir una infraestructura en la nube robusta, segura y eficiente sin depender de plataformas comerciales propietarias complejas. La verificación mediante Postman y Newman respaldó al 100% el correcto funcionamiento de los flujos de comunicación y las reglas de negocio del sistema.

Repositorio del proyecto

https://github.com/MarkUmes/FinalArqui1